1. INTRODUCTION

The main objective of the project is to develop an intelligent, camera-based **Smart Park System** that automatically detects the availability of parking spaces using computer vision. This system is designed to monitor six parking slots (three in each of two rows) and provide real-time status updates to users via a simple and responsive web interface.

In many urban areas and institutional premises, the problem of vehicle congestion and inefficient use of parking space has become a daily challenge. Traditional parking systems are either manual or sensor-based, which are expensive and harder to maintain. This project offers a low-cost and scalable alternative using **OpenCV** for image processing and a **Flask-based web interface** to display parking availability without streaming the live camera feed.

The camera captures live video of the parking area and divides the frame into predefined zones (Regions of Interest) that represent each slot. By analyzing these zones, the system detects whether a slot is occupied or free. The slot status is then transmitted to a hosted website, which visually displays which slots are available and which are occupied.

This project is especially useful for institutions, small shopping complexes, or residential societies where parking management is currently done manually or without proper space tracking. The Smart Park System automates the process, reduces human effort, minimizes search time for parking, and improves space utilization.

2. FEASIBILITY STUDY

Feasibility is the determination of whether or not a project is worth doing. The process followed in making this determination is called a Feasibility Study. Once it has been determined that the project is feasible keeping the benefit of the Organization in mind, the analyst can go ahead and prepare the project specification, which finalizes the project requirements. Different tests of feasibility are studied during the investigation. The main of them are –

- Technical feasibility
- Economical feasibility
- Operational feasibility

2.1 TECHNICAL FEASIBILITY:

technology, hardware, and software are available to implement the proposed system effectively.

- The Smart Park System uses **OpenCV**, a popular computer vision library, and **Python**, both of which are open-source and highly compatible with modern development environments.
- The camera used for monitoring parking slots can be a basic USB camera or an IP camera, which are easily accessible.
- The web interface is built using **Flask**, a lightweight Python web framework, along with HTML, CSS, and JavaScript—technologies that are standard and reliable.
- The system is technically feasible because all tools and technologies required are readily available and widely supported.

2.2 ECONOMICAL FEASIBILITY

Economic feasibility involves the cost-benefit analysis of the project—ensuring that the cost of development is justified by the benefits it provides.

- The project uses **free and open-source technologies**, reducing licensing and development costs.
- Basic hardware such as a camera and computing device (like a laptop or Raspberry Pi) is sufficient to run the system.
- The system reduces the need for manual monitoring and can potentially save on manpower costs.
- Overall, the Smart Park System is **cost-effective and economically viable** for small to medium-scale parking environments.

2.3 OPERATIONAL FEASIBILITY

Operational feasibility focuses on how well the system will work when deployed in a real-world environment.

- The system is user-friendly and requires **minimal training** to operate.
- Slot statuses are clearly displayed on a web page, which can be accessed from any device with internet access.
- The system works in real time, providing accurate and up-to-date parking information to users.
- Operationally, the system enhances the parking experience by reducing the time spent searching for a space and ensuring optimal space usage.
- The project is **operationally feasible** as it addresses a practical problem with a simple and effective solution.

3. LITERATURE SURVEY

3.1 EXISTING SYSTEMS

The existing parking systems are either **manual** or **sensor-based**, with most small institutions or commercial spaces relying on **manual monitoring** by guards or security staff. These systems often involve the staff tracking which parking slots are available and guiding vehicles manually, or writing records in a register.

Such methods are inefficient, error-prone, and time-consuming. They also lead to **driver frustration**, especially during peak hours, as users must drive around to find an available slot.

3.1.1 DRAWBACKS

- **High chances of human error** in tracking and guiding vehicles
- **No real-time updates** for users searching for slots
- **No centralized dashboard** for slot monitoring
- **Redundancy** and **inconsistency** in manual records
- **Wastage of time and resources** due to poor space utilization
- **No remote access** to parking availability data
- **Difficult to scale** for larger lots or multi-level facilities

3.2 PROPOSED SYSTEM

The proposed **Smart Park System** is an **automated, AI-enabled application** designed to replace outdated, manual parking management systems. It leverages a **camera and OpenCV** to monitor six parking slots and determine whether each is vacant or occupied.

Instead of relying on sensors or manual logs, the system uses **image processing** to detect vehicles and update slot statuses on a **dynamic website** in real time. The system does not stream the camera feed online, which ensures better performance and privacy. The web page simply shows the current status (available/occupied) of each slot.

This system can be deployed in a variety of places such as colleges, hospitals, residential complexes, and small malls to improve parking efficiency and reduce congestion.

3.2.1 REQUIREMENT SPECIFICATION

The proposed software application includes the following features:

- Automated detection of vehicles using a **single camera**
- **Real-time slot monitoring** without sensor hardware
- Web-based interface using **Flask + HTML/CSS**
- **Modular and scalable** backend logic
- Display of current status of all six slots on a dashboard
- Future scope for **notifications** and **slot booking**

3.2.2 NEED FOR COMPUTERISATION

With increasing demand for automated infrastructure, computerization of parking systems has become essential. Manual approaches are no longer sustainable in modern-day environments where real-time data is crucial.

This system eliminates the need for manual supervision and helps utilize every slot efficiently. It offers automation with minimal setup and ensures faster, smarter, and more convenient parking management.

3.2.3 BENEFITS OF THE PROPOSED SYSTEM

- Reduces manpower by eliminating manual supervision
- Accurate and real-time status of each parking slot
- Simple, user-friendly interface accessible from browser
- Cost-effective solution using open-source tools
- Easily extendable to multiple locations and floors
- Better parking experience for users
- Environmentally friendly by reducing fuel used to search for slots

4. REQUIREMENT SPECIFICATIONS

4.1 HARDWARE REQUIREMENT FOR THE APPLICATION

Component	Specification
Processor	Intel Core i3 or above / Raspberry Pi 4
RAM	Minimum 4 GB
Hard Disk	64 GB SSD or higher
Camera	USB camera / IP camera
Internet	Required for web access (optional for local use)

The system can be deployed on any personal computer or Raspberry Pi device capable of running Python and handling real-time image processing.

4.2 SOFTWARE REQUIREMENT FOR THE APPLICATION

This project is developed using open-source technologies for both backend and frontend components. Below are the detailed software requirements:

Component	Specification
Operating System	Windows 10 / Linux Ubuntu
Programming Language	Python 3.9+
Web Framework	Flask (Python micro-framework)
Computer Vision Library	OpenCV (cv2)
Frontend Technologies	HTML, CSS, JavaScript
IDE/Editor	Visual Studio Code / PyCharm
Database (optional)	SQLite or JSON for logging (if needed)
Browser	Google Chrome, Firefox, or Edge

5. SYSTEM DESIGN

System design develops the architectural details required to build a system or product. The system design process encompasses the following activities:

- Partition the analysis model into subsystems
- Identify concurrency that is dictated by the problem
- Allocate subsystem to processor and tasks
- Choose a basic strategy for implementing data management
- Identify global resources and the control mechanisms required to access them
- Design an appropriate control mechanism for the system, including task management
- Consider how boundary conditions should be handled
- Review and consider trade-off

Software design is an interactive process through which requirements are translated into a “Blue Print” for constructing the software. Initially the blue print depicts a holistic view of software. That is the design is represented at a high level of abstraction a level that can be traced to the specific system objective and more detailed data, functional and behavioral requirements.

Throughout the design process the quality of the evolving design is assessed with a series of formal technical reviews.

- The design must implement the entire explicit requirement contained in the analysis model and it must accommodate all of the implicit requirements desired by the customer.
- The design must be readable, understandable guide for those who generated code and for those who test and subsequently support the software.
- The design should provide a complete picture of the software addressing the data, functional and behavioral domain from an implementation perspective.

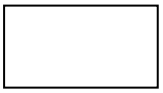
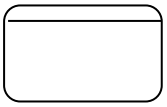
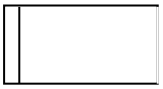
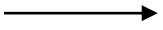
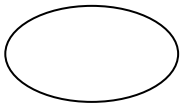
5.1 DATA FLOW ANALYSIS

The data flow analysis is a study of how and what the data flows through the system. It is done with the help of Data Flow Diagram (DFD), which depicts the entities involved in the system, data input by the entities, processes the input data undergoes and data store where the data is stored.

5.1.1 CONTEXT LEVEL DATA FLOW DIAGRAM

The context level DFD shows the interaction of the system with the external environment. It only depicts the inputs to the system and output from the system but it does not show how they are transformed. The functional aspects of the system are not covered under context level DFD.

Notations used in DFD

Sl. No.	Symbol	Meaning
1		Entity
2		Process
3		Data Store
4		Data flow with direction
5		User Screen

The data flow diagram is particularly useful for showing high-level functionality of a system and its break down into smaller functional units.

6. DETAIL DESIGN

Detailed design covers the study of the major entities involved in the system, their attributes and relationships, how they can be transformed into normalized tables and what is the dependency among the tables. While system definition is design oriented, detailed design is implementation oriented. By defining logical and physical structure of the database, detailed design guides the implementation phase.

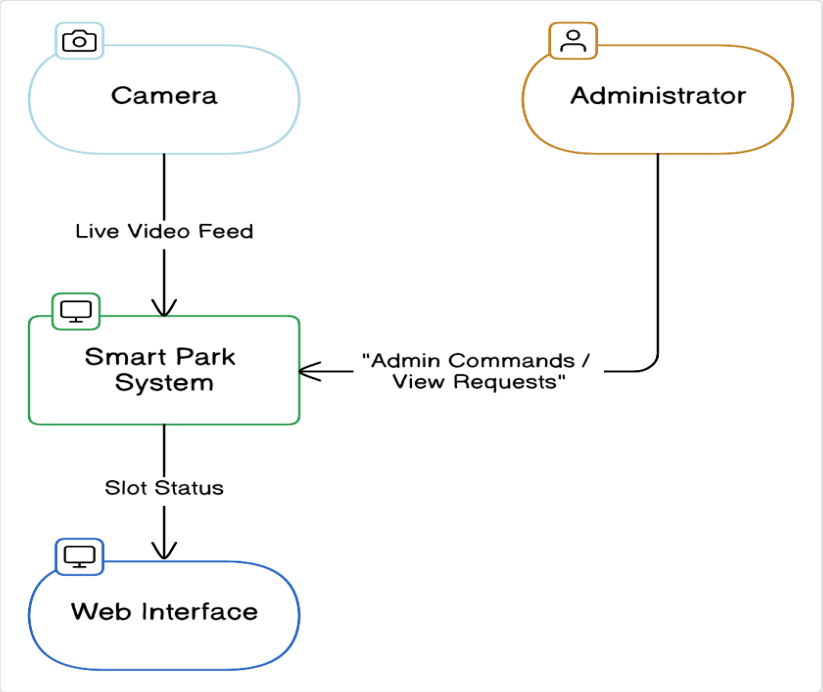
6.1 CONTEXT LEVEL DATA FLOW DIAGRAM

The context level DFD shows the interaction of the system with the external environment. It only depicts the inputs to the system and output from the system but it does not show how they are transformed. The functional aspects of the system are not covered under context level DFD.

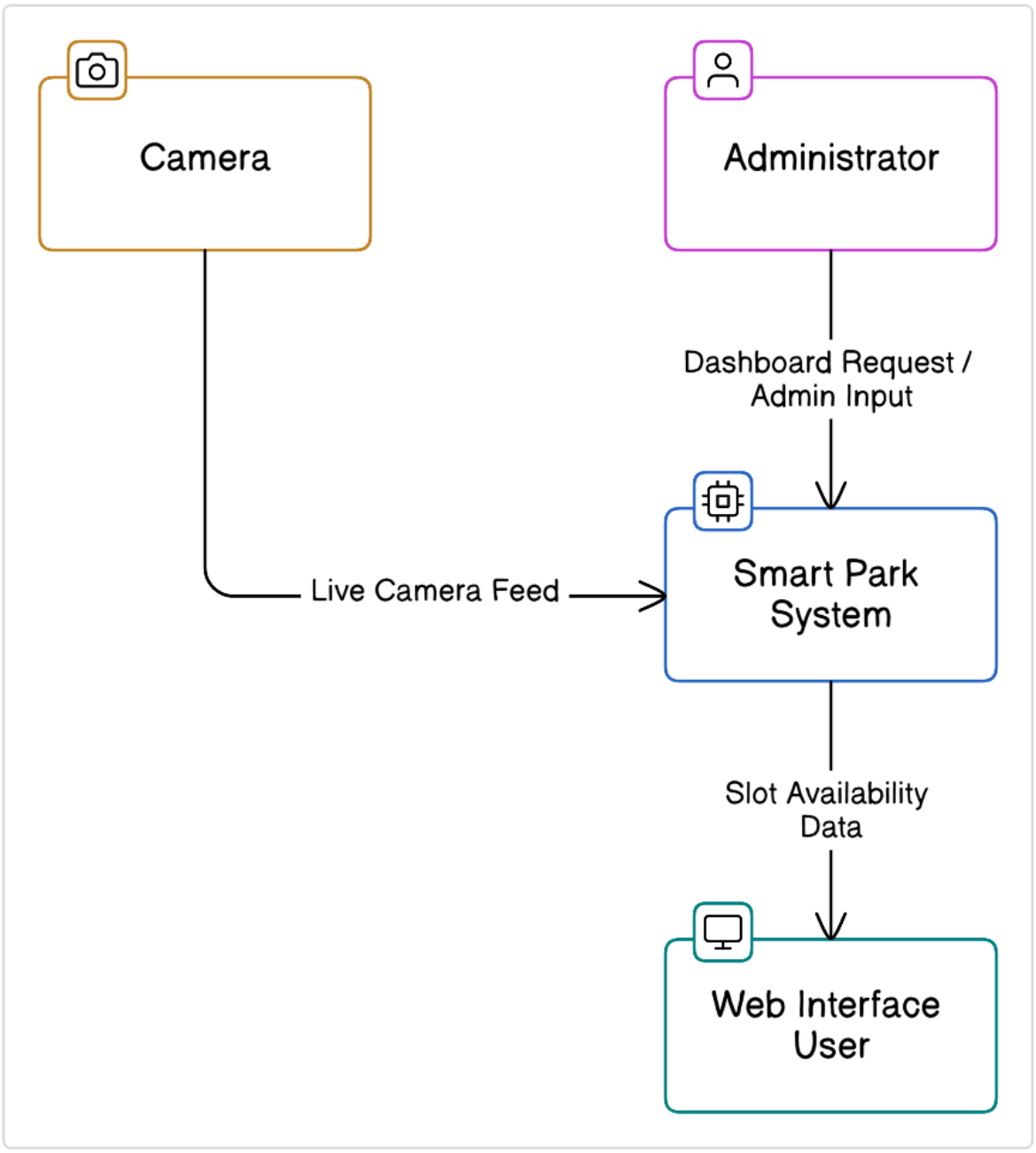
Data Flow Diagram

Context Level Diagram

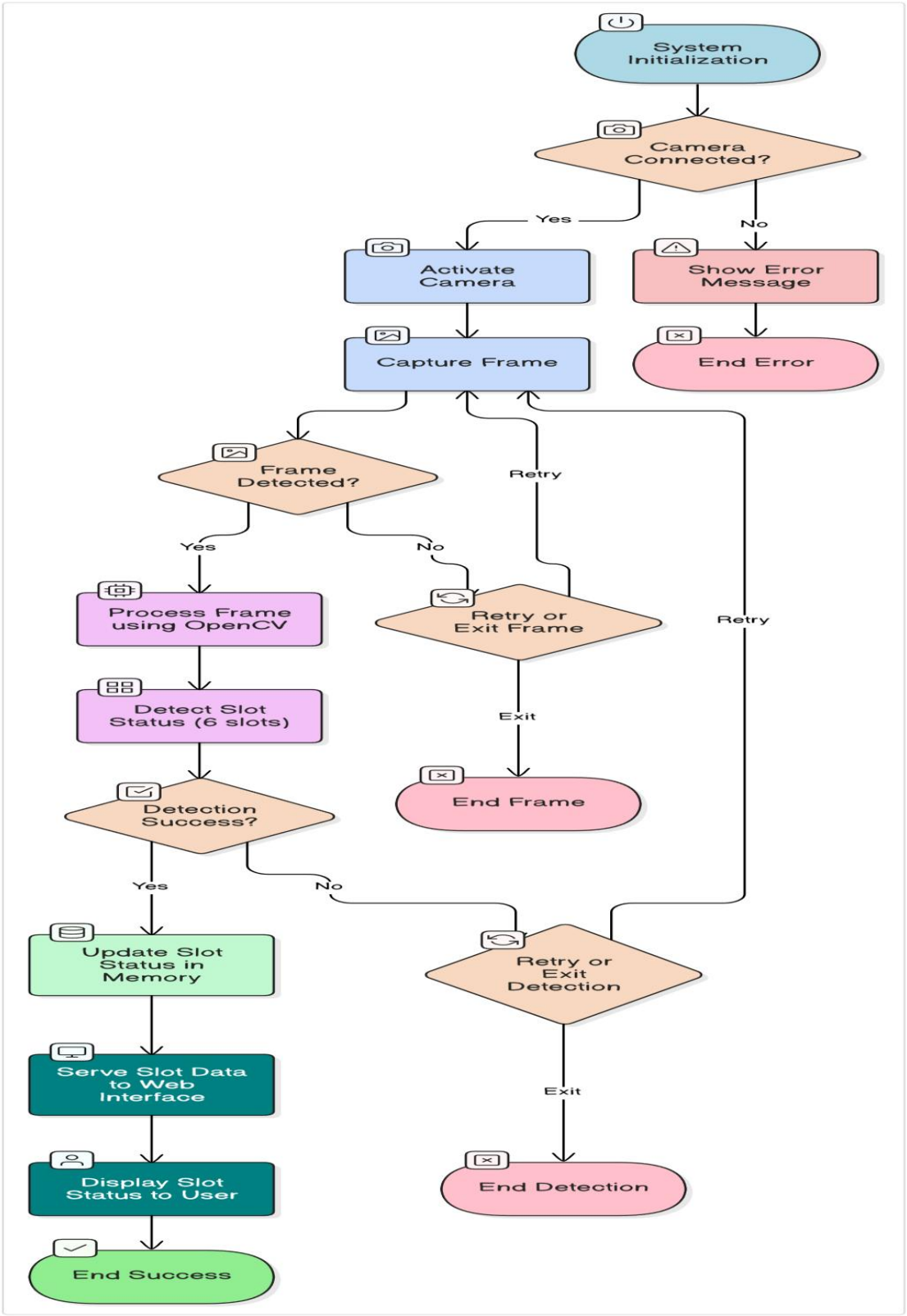
Level -0



Context Diagram

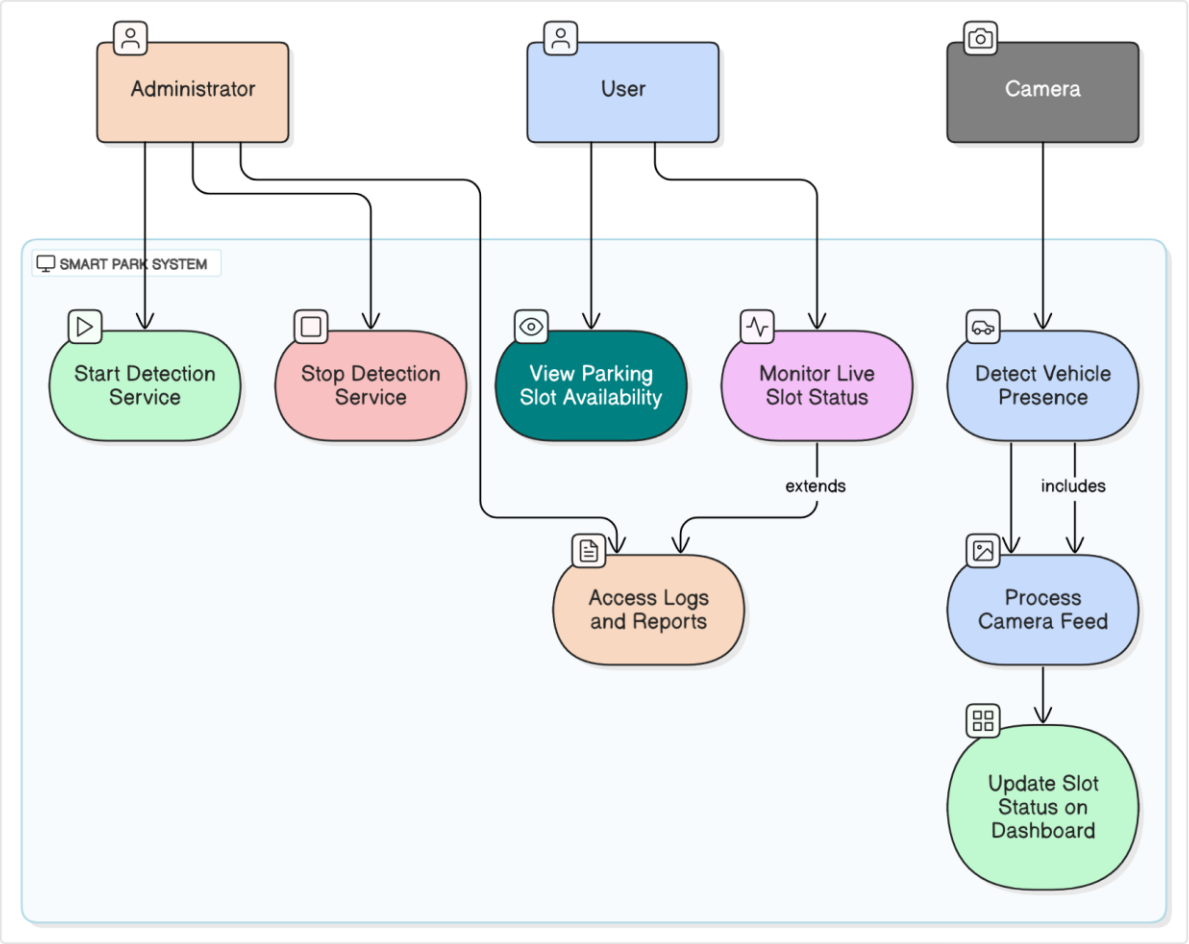


Control Flow Diagrams



6.2 USE CASE DIAGRAMS

This Use case provides the information about the user and module related information in this system, the use case shows how the user and the module role are defined and flow of the work.



6.3 DATA FLOW ANALYSIS

The data flow analysis is a study of how and what the data flows through the system. It is done with the help of Data Flow Diagram (DFD), which depicts the entities involved in the system, data input by the entities, processes the input data undergoes and data store where the data is stored.

6.3.1 CONTEXT LEVEL DATA FLOW DIAGRAM

The context level DFD shows the interaction of the system with the external environment. It only depicts the inputs to the system and output from the system but it does not show how they are transformed. The functional aspects of the system are not covered under context level DFD.

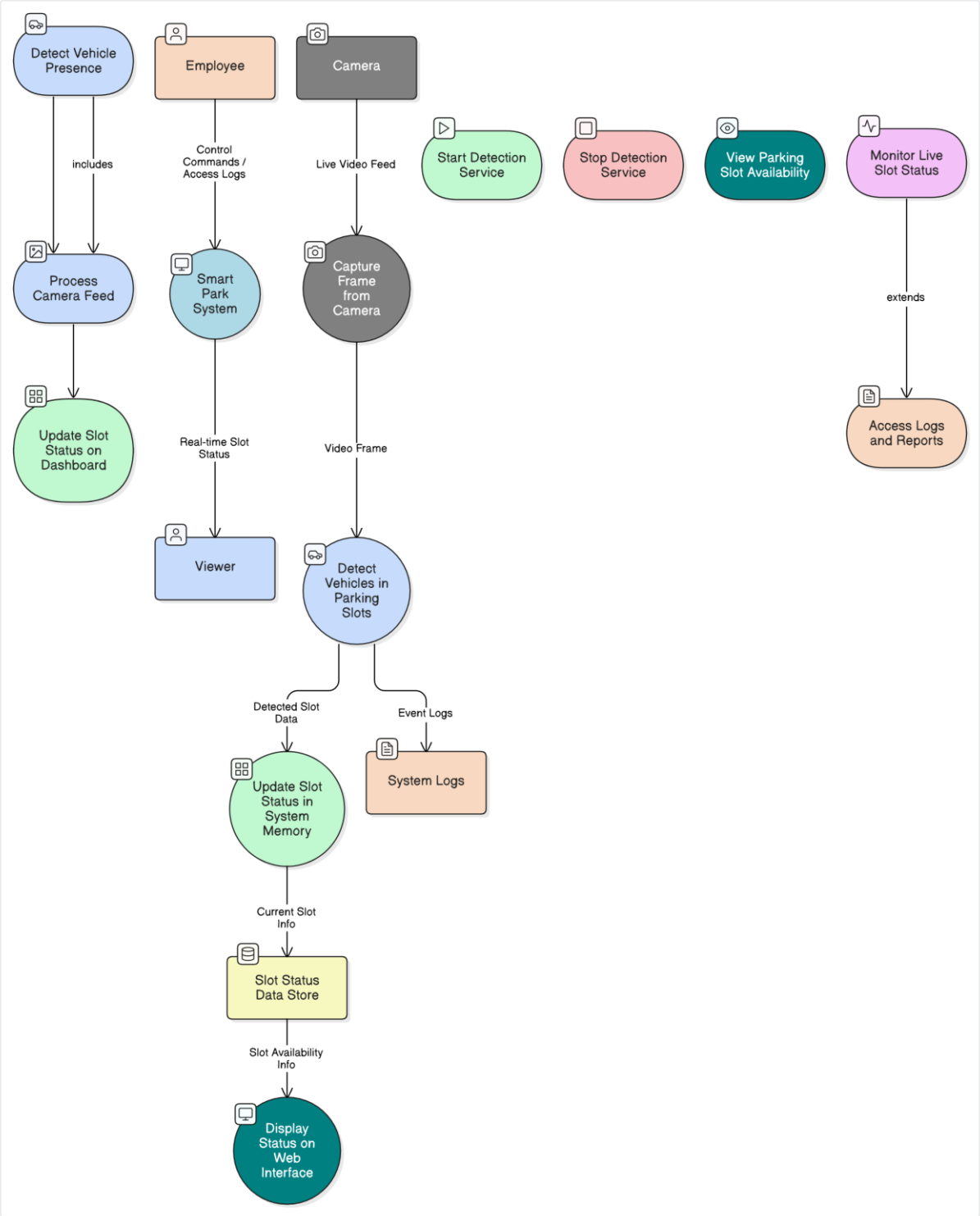
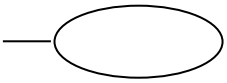
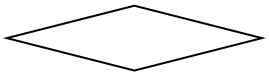
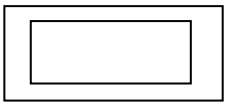
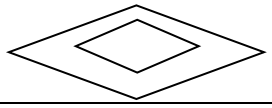
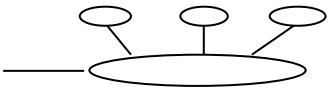
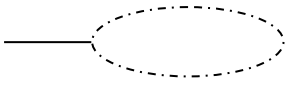
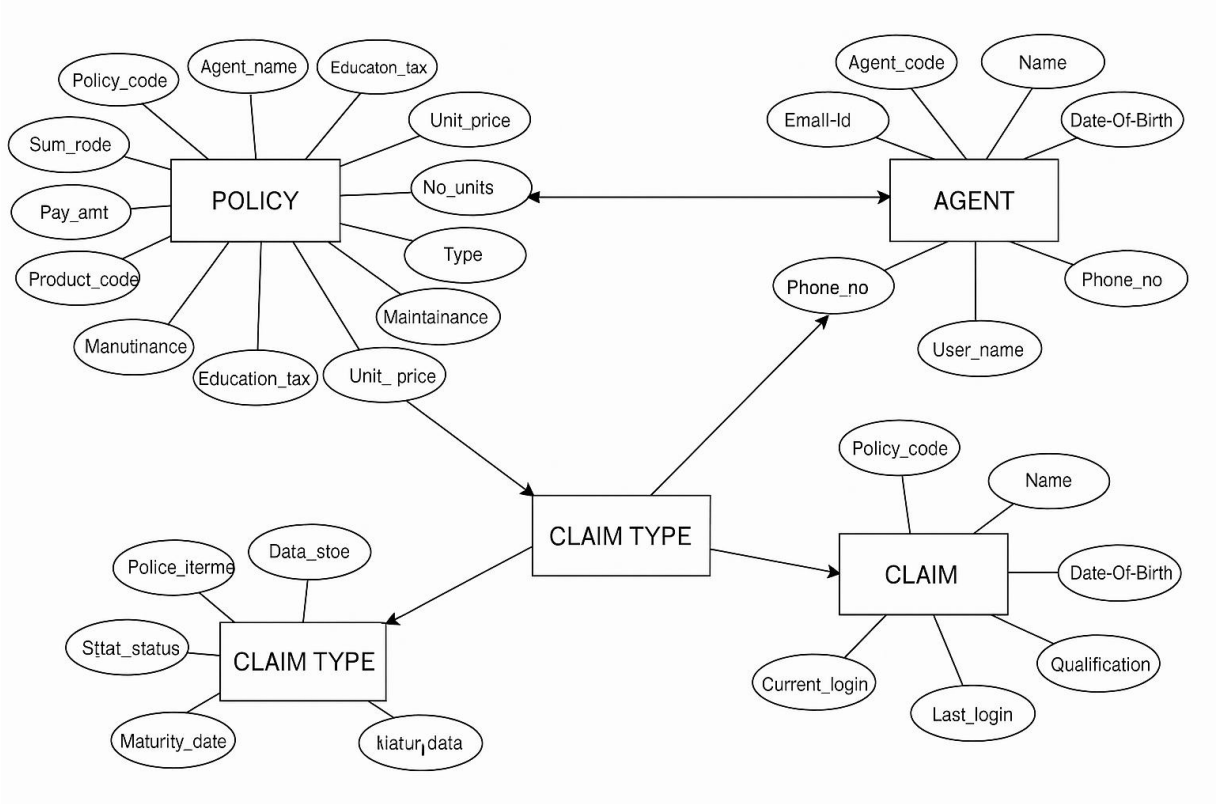


Figure 6.3.1 Context Level DFD for Smart Parking System.

6.4 THE ER SCHEMA

The overall logical structure of the database can be expressed graphically by an E-R diagram shows the entities, their attributes, relationship and cardinality among the entities. Following are some of the notations used in E-R diagram.

Sl.No	Symbol	Meaning
1		Entity
2		Attribute
3		Key Attribute
4		Relationship
5		Relations
6		Weak Entity
7		Multi Valued Attribute
8		Identifying Relationship
9		Composite Attribute
10		Derived Attribute



6.4.1 DETAILED DESIGN OF THE MODULES

This section focuses on the internal logic of the modules specified in the system definition. The detailed design of the modules helps in generating the algorithms for the system. Here we present the in-detail study of modules used in the application and processes that the modules encompass.

Module: Administrator

Process: Login

Action: Click on the Login button

Input: Username and Password

Tables used: Users

Stored Procedures/ Triggers used: None

Brief description: When the administrator clicks the Login button, the system fetches the username and password, encrypts the input, and compares it with the stored encrypted credentials in the Users table. If the data matches, the session is started for the admin. Otherwise, an error message is shown indicating invalid login credentials.

7. IMPLEMENTATION

Implementation is the stage where the theoretical design is turned into a working system. Once the design is complete, most of the major decisions about the system have been made. The goal of coding phase is to translate the design of the system into code in a given programming language. For a given design, the aim in this phase is to implement the design in the best possible manner.

The coding phase affects both testing and maintenance profoundly. Since the testing and maintenance costs of software are much higher than the coding cost, the goal of the coding should be to reduce the testing and maintenance effort. Hence, during coding the focus should be on developing the programs that are easy to read and understand, and not simply on developing programs that are easy to write.

7.1 PROCESS FLOW (ACTIVITY) DIAGRAM

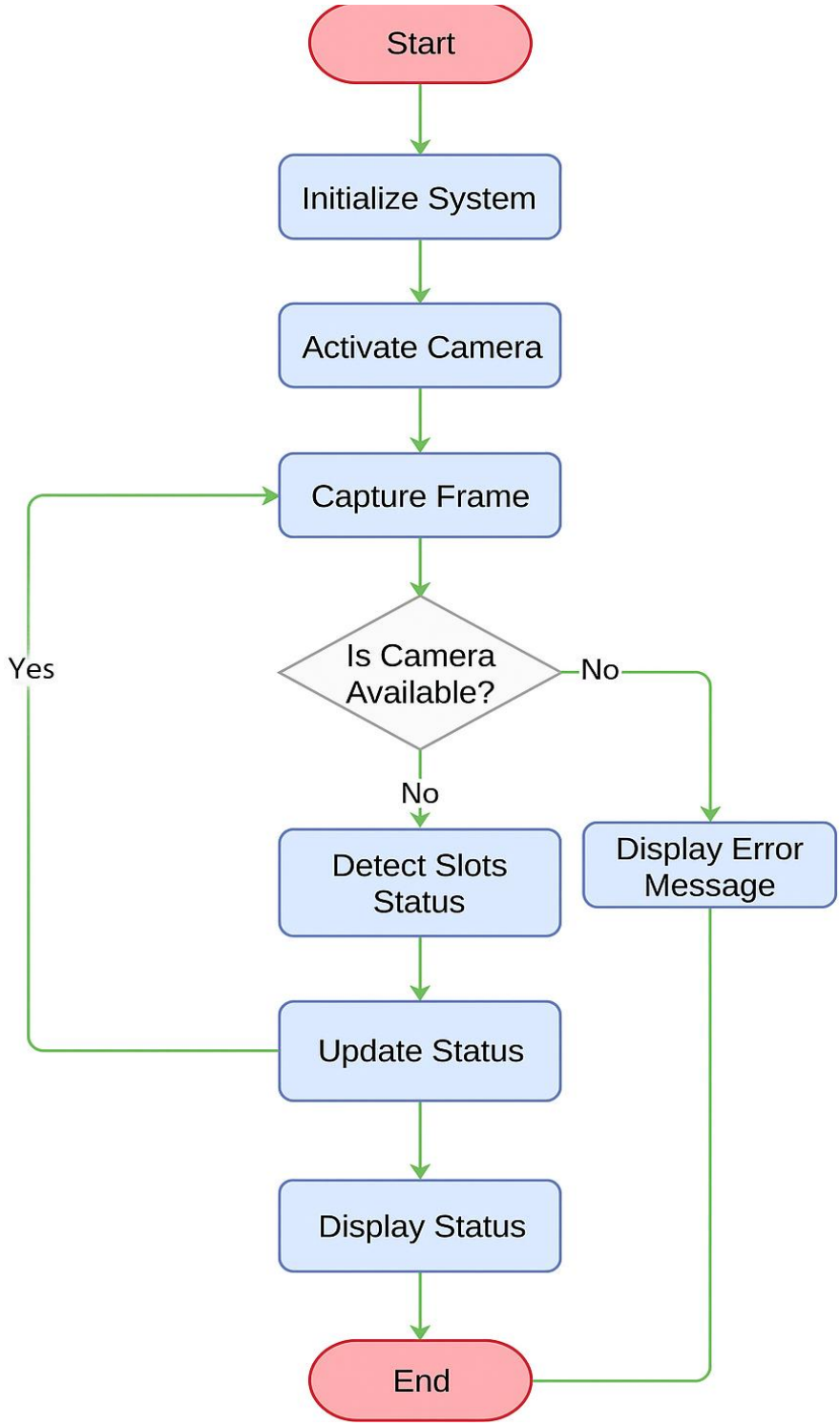


Figure 7.1 Activity diagram depicting the process flow of the system

8. TOOLS USED

The Development Of The Smart Park System Involved The Use Of Multiple Open-Source Software Tools And Libraries. Each Tool Was Chosen To Match The System's Requirements, Ensure Compatibility, And Reduce The Overall Development Cost.

8.1 PYTHON

Python is the core programming language used to develop the system logic. It is simple, readable, and widely supported, especially in fields like image processing, AI, and backend development.

8.2 OPENCV (OPEN SOURCE COMPUTER VISION LIBRARY)

OpenCV is an open-source library used for **real-time image processing**. It handles:

- Capturing video feed from the camera
- Processing individual frames
- Detecting the presence of vehicles in parking slots
- Drawing indicators (like bounding boxes or slot markers)

8.3 FLASK

Flask is a lightweight Python web framework used to build the **backend server**. It handles routing, data passing between detection logic and the web interface, and API endpoints for real-time status updates.

8.4 HTML / CSS / JavaScript

These standard frontend technologies were used to create the **user interface (UI)**:

- **HTML**: Structures the web page
- **CSS**: Styles the slot indicators (e.g., red/green status)
- **JavaScript**: Enables live updates and dynamic content rendering

8.5 Visual Studio Code

VS Code served as the **primary Integrated Development Environment (IDE)** for writing and testing the code. It supports Python, HTML/CSS/JS, and integrates easily with GitHub and extensions like Live Server.

9. TESTING AND RESULTS

9.1 TESTS CONDUCTED

9.1.1 TESTS CONDUCTED IN THE ACTIVITIES OF THE SLOT DETECTION TESTING AS FOLLOWS :

Test Case No	Module	Input	Expected Output	Test Result
1	Slot Detection	Car present in Slot 1	Status: Occupied	Pass
2	Slot Detection	Slot 2 is empty	Status: Available	Pass
3	Slot Detection	Multiple cars in frame	Correct slot detection	Pass
4	Slot Detection	Poor lighting condition	Accurate status or fail-safe	Pass

9.1.2 TESTS CONDUCTED IN THE ACTIVITIES OF THE WEB INTERFACE TESTING AS FOLLOWS :

Test Case No	Module	Input	Expected Output	Test Result
1	Web UI	Open dashboard	Display all slots	Pass
2	Web UI	Slot 3 changes to occupied	Update shown on screen	Pass
3	Web UI	Refresh browser	Persistent status display	Pass
4	Web UI	Disconnect camera temporarily	Display last known state	Pass

9.2 DETAILED TEST PLAN

System testing is an important stage in the system development life cycle. It is essential for thorough checking and fault finding for different sets of conditions. The developed product is tested for various inputs and it should be error free and should work normal under normal condition as long as user gives proper inputs. There are many ways of testing the reliability, completeness, correctness, robustness and maintainability of the software package**.**

The testing commences with the test plans and terminates with the acceptance testing. Testing is applied at different levels in the software development Life cycle. The focus of all testing is to find errors, but different types of errors, are errors are looked for different levels. There are many ways of testing as follows:

- Unit Testing
- Integration Testing
- System Testing
- Validation Testing

9.2.1 TYPES OF TESTING

Unit Testing

The unit testing focuses verification effort on the smallest unit of the software design module. In unit testing different modules are tested against the specifications produced during the design of module. Here important control parts are tested o uncover the errors within the boundary of the module. The relative complexity of the tests and results is limited by the constrained scope established for the unit testing.

The testing is done considering each unit available in our website, with respect to every recent update.

Integration Testing

In the integration testing the testing modules are combined in to subsystems, which are then tested. Integration testing refers to the retesting components/functionality of the system to ensure that they function properly even after and change has been made to parts of the system. As detects are discovered in a component, modification are made to correct them.

System Testing

In system testing the entire software system is tested. The reference document for this process is requirement document. It also tests to find the discrepancies between the system and the original objective, current specification and system documentation. The entire system is checked to correct deviation to achieve correctness.

Validation Testing

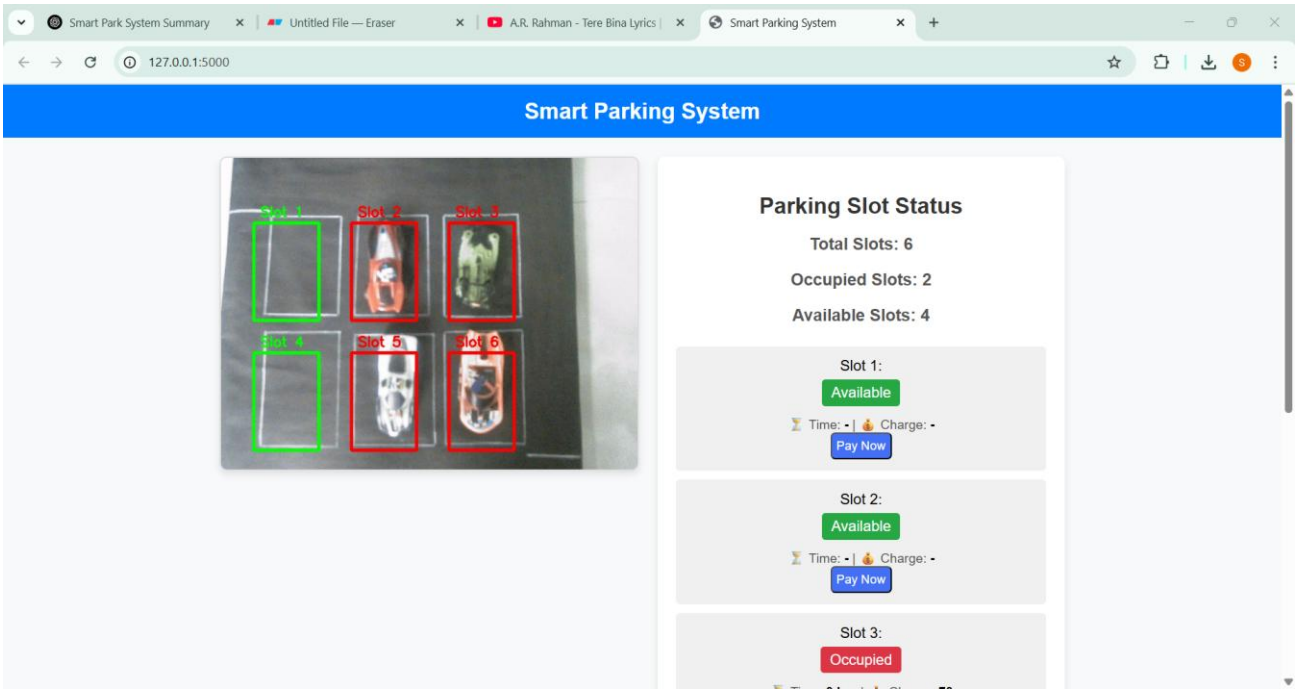
Software validation is achieved through the black box tests that demonstrate the conformity with the requirement. Both plan and procedures are designed to ensure that all the functional are requirements are satisfied.

After each validation test has been conducted, one of the two possible conditions exists

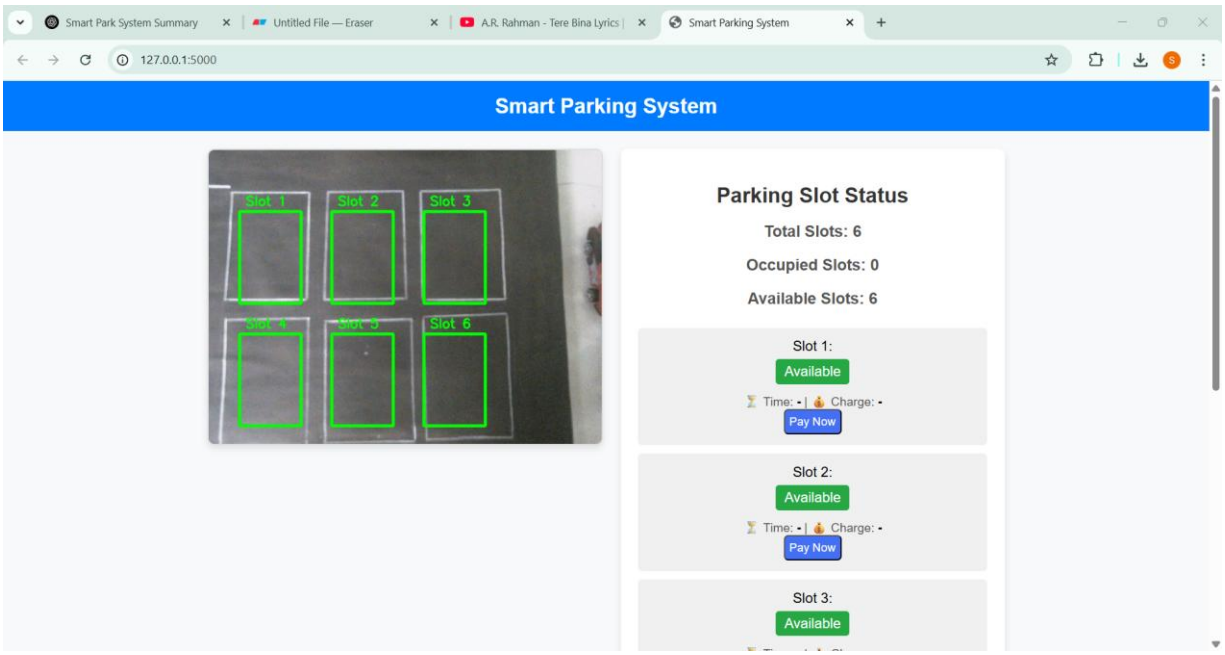
- The function or the performance characteristics confirm to specification and are accepted.
- A deviation from specification is uncovered and a deficiency list is created.

10. SNAPSHOTS

The snapshots make the users familiar with the interface of the application and serve as user manual. Here we can see some important sample screen shots of the application.



Snapshot 10.1 Slot Details when cars filled



Snapshot 10.2 Slot Details when there's no cars

11. CONCLUSION

The **Smart Park System** was developed to provide a simple, cost-effective solution to everyday parking challenges using a combination of **computer vision** and **web technologies**. The system accurately detects the availability of six parking slots using OpenCV and displays the real-time status on a responsive, user-friendly web interface.

During implementation, the system demonstrated **high detection accuracy**, reliable performance, and ease of use. The primary objective—to automate parking space monitoring without expensive sensors—was fully achieved using affordable hardware and open-source software.

The system minimizes human intervention, improves space utilization, and enhances the parking experience, particularly in institutions, small malls, and residential complexes.
This project also lays a strong foundation for future improvements and real-world deployment at scale.

12. FUTURE ENHANCEMENTS

While the current version of the Smart Park System is fully functional, several enhancements can be made to improve performance, scalability, and user convenience:

- **License Plate Detection**: Integrate OCR to detect and log vehicle registration numbers.
- **Mobile Application**: Develop an Android/iOS app for viewing slot status and receiving alerts.
- **Cloud Hosting**: Move the dashboard and detection service to the cloud for broader accessibility.
- **Notification System**: Add email/SMS alerts when slots become available or full.
- **Payment Integration**: Include a system for booking and paying for slots through UPI/wallets.
- **Multi-level Parking Support**: Extend system to monitor and manage multi-floor parking spaces.

These enhancements can turn the Smart Park System into a fully scalable **smart parking platform**.

BIBLIOGRAPHY

Books:

- **OpenCV Documentation** – https://docs.opencv.org
- **Flask Web Framework** – https://flask.palletsprojects.com
- **Python Official Docs** – https://docs.python.org/3
- **HTML & CSS Reference** – MDN Web Docs – https://developer.mozilla.org
- **GitHub Repositories** – Various project inspirations and samples